Lập trình mạng · Chương 5

Độ tin cậy: backoff, circuit breaker và health check

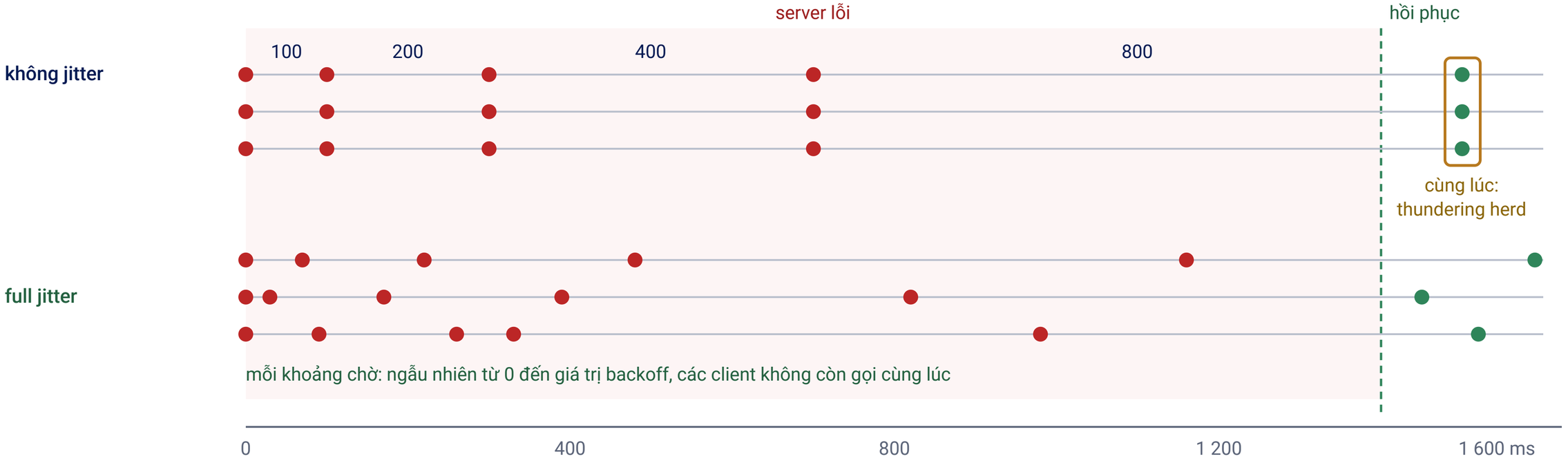
TS. Đặng Ngọc Hùng

Gọi lại liên tục làm server đang lỗi tệ thêm

1. Server tra cứu lỗi vài giây: mọi client **retry** với khoảng chờ cố định
2. Retry dồn thêm tải lên server đang lỗi, rồi đồng loạt quay lại lúc server vừa hồi phục
3. Bộ cân bằng tải cần biết server nào đang lỗi để tạm không chuyển request tới

> **Câu hỏi của phần:** chờ bao lâu trước khi gọi lại, lúc nào nên tạm ngừng gọi, và server báo trạng thái thế nào?

Exponential backoff giãn dần khoảng chờ; jitter tránh gọi lại cùng lúc



Exponential backoff nhân đôi khoảng chờ, có trần; **jitter** chọn ngẫu nhiên trong khoảng đó để tránh **thundering herd**.

Circuit breaker: ba trạng thái CLOSED, OPEN, HALF_OPEN



Circuit breaker ngừng gọi khi lỗi liên tiếp (OPEN), hết thời gian chờ thì cho lời gọi thử (**half-open**), thử thành công thì gọi lại bình thường.

Circuit breaker trước server lỗi từ giây 1,0 đến giây 3,4

Java

```
synchronized boolean allowRequest(long now) {
    if (state == State.OPEN && now - openedAt >= openMillis) {
        state = State.HALF_OPEN;  // hết thời gian chờ: cho một lời gọi thử
    return state != State.OPEN;  // OPEN: từ chối ngay, không gửi request
synchronized void onFailure(long now) {
    if (state == State.HALF_OPEN || failures >= failureThreshold) {  // lời gọi thử lỗi, hoặc đủ ngưỡng lỗi liên tiếp
        state = State.OPEN;
```

Kết quả chạy

```
 0.4 s  CLOSED     200      -> CLOSED     health 200
 0.8 s  CLOSED     200      -> CLOSED     health 200
 1.2 s  CLOSED     503      -> CLOSED     health 503
 1.6 s  CLOSED     503      -> CLOSED     health 503
 2.0 s  CLOSED     503      -> OPEN       health 503
 2.4 s  OPEN       rejected -> OPEN       health 503
 2.8 s  OPEN       rejected -> OPEN       health 503
 3.2 s  HALF_OPEN  503      -> OPEN       health 503
 3.6 s  OPEN       rejected -> OPEN       health 200
 4.0 s  OPEN       rejected -> OPEN       health 200
 4.4 s  HALF_OPEN  200      -> CLOSED     health 200
 4.8 s  CLOSED     200      -> CLOSED     health 200
12 calls: 8 requests sent, 4 rejected by the breaker
```

Health check: server tự báo trạng thái cho bộ cân bằng tải

```java
http.createContext("/health", this::health);
boolean up = !down();  // kiểm tra cả phần phụ thuộc (ở đây: kho dữ liệu)
send(ex, up ? 200 : 503, up ? "{\"status\":\"UP\"}" : "{\"status\":\"DOWN\"}");
```

- `GET /health`: 200 kèm `UP`, hoặc 503 kèm `DOWN`
- Kiểm tra cả phần phụ thuộc: tiến trình còn chạy chưa có nghĩa là phục vụ được
- Bộ cân bằng tải gọi định kỳ, tạm bỏ server `DOWN` khỏi danh sách

Backoff không có jitter: mọi client quay lại cùng lúc

```java
long exp = Math.min(capMillis, baseMillis << attempt);  // 100, 200, 400, 800… không quá cap
return jitter == null ? exp : (long) (jitter.nextDouble() * exp);  // full jitter: ngẫu nhiên trong [0, exp)
```

Kết quả chạy

	requests	peak / 100 ms after recovery	all OK after
fixed 100 ms	1261	85	1531 ms
backoff	500	100	1608 ms
backoff+jitter	647	14	2896 ms

- Không jitter: cả 100 client quay lại trong cùng 100 ms khi server vừa hồi phục
- Retry thao tác không idempotent vẫn cần **request ID**

Câu hỏi 1

Circuit breaker đang ở trạng thái OPEN và chưa hết thời gian chờ. Client gọi tra cứu. Điều gì xảy ra?

A. Lời gọi bị từ chối ngay, không có request nào được gửi tới server.

B. Request vẫn được gửi, nhưng kết quả bị bỏ qua.

C. Lời gọi chờ đến khi breaker chuyển sang CLOSED.

D. Breaker chuyển ngay sang HALF_OPEN để gọi thử.

Câu hỏi 1

Circuit breaker đang ở trạng thái OPEN và chưa hết thời gian chờ. Client gọi tra cứu. Điều gì xảy ra?

A. Lời gọi bị từ chối ngay, không có request nào được gửi tới server.

B. Request vẫn được gửi, nhưng kết quả bị bỏ qua.

C. Lời gọi chờ đến khi breaker chuyển sang CLOSED.

D. Breaker chuyển ngay sang HALF_OPEN để gọi thử.

OPEN từ chối ngay để server có thời gian hồi phục và bên gọi không phải chờ timeout. Chỉ khi hết thời gian chờ, breaker mới chuyển sang HALF_OPEN cho một lời gọi thử.

Câu hỏi 2

100 client cùng dùng exponential backoff 100, 200, 400, 800 ms, không có jitter, sau một sự cố chung. Vì sao nên thêm jitter?

A. Để giảm tổng số request còn một nửa.

B. Để các client không gọi lại cùng thời điểm, tránh dồn tải lên server vừa hồi phục.

C. Để client nhận phản hồi nhanh hơn.

D. Để server biết request nào là retry.

Câu hỏi 2

100 client cùng dùng exponential backoff 100, 200, 400, 800 ms, không có jitter, sau một sự cố chung. Vì sao nên thêm jitter?

A. Để giảm tổng số request còn một nửa.

B. Để các client không gọi lại cùng thời điểm, tránh dồn tải lên server vừa hồi phục.

C. Để client nhận phản hồi nhanh hơn.

D. Để server biết request nào là retry.

Không có jitter, các client có cùng lịch gọi lại và quay lại cùng lúc (thundering herd). Jitter rải các lần gọi ra; nó không nhằm giảm tổng số request.

Tổng kết

1. **Exponential backoff** giãn dần khoảng chờ; **jitter** rải lần gọi lại, tránh **thundering herd**

2. **Circuit breaker**: CLOSED, OPEN, **half-open**; tự ngừng gọi khi server lỗi, tự gọi lại khi đã hồi phục

3. **Health check** kiểm tra cả phần phụ thuộc; bộ cân bằng tải dựa vào đó để chuyển request

Từ khóa cần nhớ exponential backoff · jitter · circuit breaker · half-open · health check · thundering herd

Mở rộng

Chủ đề nâng cao

- Thư viện Resilience4j
- Bulkhead và rate limiter

Bài tập mở rộng

Thay circuit breaker tự viết bằng Resilience4j; cấu hình ngưỡng lỗi 50% trong 10 lời gọi.

Tiếp theo

Bài toán ứng dụng: giám sát cảm biến với MQTT và CoAP